A New Perspective on Viewing

John Hilton
September 2009
Revised January 2026

Introduction

3D computer graphics viewing is a straightforward mapping of graphical items in a 3D 'view' volume to a 2D image. Yet there is no standard way to specify the view, and there are a wide range of different viewing implementations currently in use. Closely tied in to viewing parameters is 3D interaction, and there are likewise many different 3D interaction implementations often providing the same basic functionality. Users are forced to learn and use different pan, zoom, and spin techniques for each 3D program they use. As every 3D program must reinvent this functionality, why hasn't a standard emerged after so many decades? This lack of standardization leads to higher development and support efforts as well as stifling the proliferation of user interface features like stereo.

A standard viewing software toolkit, along with a standard motion toolkit, would benefit end users by delivering consistent and comprehensive 3D interaction across applications, and would benefit developers by reducing development time, software support, and customer support. All the various interaction techniques can be provided to satisfy the varying requirements of the different types of 3D programs and the different levels of end user expertise.

At the foundation of viewing code, there are calls made to the underlying 3D API. The popular 3D APIs, namely OpenGL® and Direct3D®, have different viewing calls. An independent low-level set of viewing parameters is needed to handle all possible types of 3D views. The viewing and motion toolkits can then be built on top of these.

A new and simpler perspective on 3D viewing (pun intended) is presented here, leading to a set of seven viewing parameters that handle all types of views found in 3D computer graphics. Code is provided for configuring OpenGL and Direct3D with these low-level viewing parameters. Sample code also shows a higher-level set of viewing parameters suitable for many common types of 3D programs.

The attached "*NewView*" demo has both OpenGL and Direct3D windows side by side showing how the viewing parameters are used to configure the respective views. Some simple mouse interaction is also provided.

Viewing Basics

Viewing maps a 3D view volume to a 2D image. Viewing involves transformation, clipping, occlusion, and rendering operations. Transformations map geometric information such as coordinates or directions to a specified coordinate system or 'space' where other operations are performed. Clipping excludes any graphical items that are external to the view volume and trims any items crossing the view volume's surface or surfaces. Occlusion hides any items or parts of items that are behind items closer to the front of the view volume. Rendering generates pixel or line plotting values based on the graphical item's geometry and on color or texture information, along with other scene information such as lighting.

A virtual world is generally defined as a tree structure that associates graphical items into collections and specifies how the items positionally relate to each other. The top level or root of the tree is usually considered the 'world' space. The relative spatial position of child items to their parent is specified by

rotate and translate transformations. Scale and skewing (a.k.a. shearing) transformations may be used to change the size and shape of graphical items. The view volume can itself be specified anywhere in the tree structure. Due to the treatment of viewing by computer graphics textbooks, most 3D programs happen to use a set of viewing parameters independent of the tree structure.

The 3D view volume can be fully specified by defining its shape, then orienting and positioning the shape relative to world space. A 'view' space is defined to specify the shape, then a rotate/translate transformation is used to locate this view space in world space.

When the 2D output image is a rectangular shape, the 3D view volume is either a rectangular prism (parallel views) or a truncated pyramid (perspective views). A truncated pyramid is called a frustum. Consider forming the view volume by either sweeping a rectangle along a straight-line segment to form a prism or scaling a rectangle about a 3D point to form a frustum. Note that the axis of the prism or frustum may or may not be normal to the rectangular cross section. For simplicity, the view volume is aligned in view space with the rectangular cross section parallel to the z = 0 plane; this makes the front and back clipping planes parallel to the z = 0 plane. The rectangular cross section's sides are aligned to the X and Y axes, so view space +X is right, and +Y is up.

Clipping calculations are simpler when the 3D view volume, transformed to clipping space, is bounded by an appropriate set of six of the x = 0, y = 0, z = 0, x = 1, y = 1, z = 1, x = -1, y = -1, and z = -1 clipping space planes. This means the view-to-clipping-space transformation typically needs to deform the rectangular prism or frustum in view space into a square prism or cube in clipping space. There are two common clipping volumes: one is a cube with opposing corners at (-1, -1, -1) and (1, 1, 1), and the other is a square prism with opposing corners at (1, -1, 0) and (1, 1, 1). The two most popular 3D graphics programming interfaces are OpenGL® and Direct3D®. OpenGL clips to the cube, and Direct3D to the square prism. Appropriate 'projection' transformations map the view volume from view space to clipping space. Note: Some older wireframe graphic systems do not clip in the depth direction, and the view volume theoretically extends to infinity.

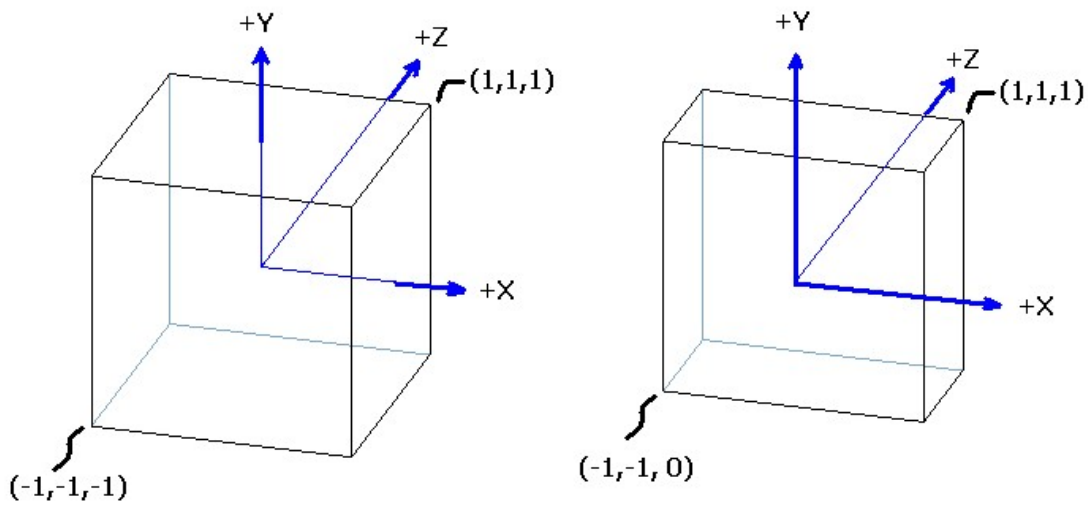


Figure 1. Clipping space view volumes.

Angular calculations, such as the angle of a surface to a light source, are required for many types of lighting and coloring operations. The view-to-clipping-space transformation does not preserve angles or distance, and so angular and distance calculations cannot be done in clipping space. View space does have correct angles so, typically, all items are transformed to view space where angular and distance lighting calculations are performed. The view space coordinates are then transformed to clipping space where clipping is done. The clipping space coordinates are finally transformed to pixel and depth buffer space where occlusion and rendering calculations are performed.

General View Volume

The general rectangular prism or frustum view volume can be specified in view space with the following seven values:

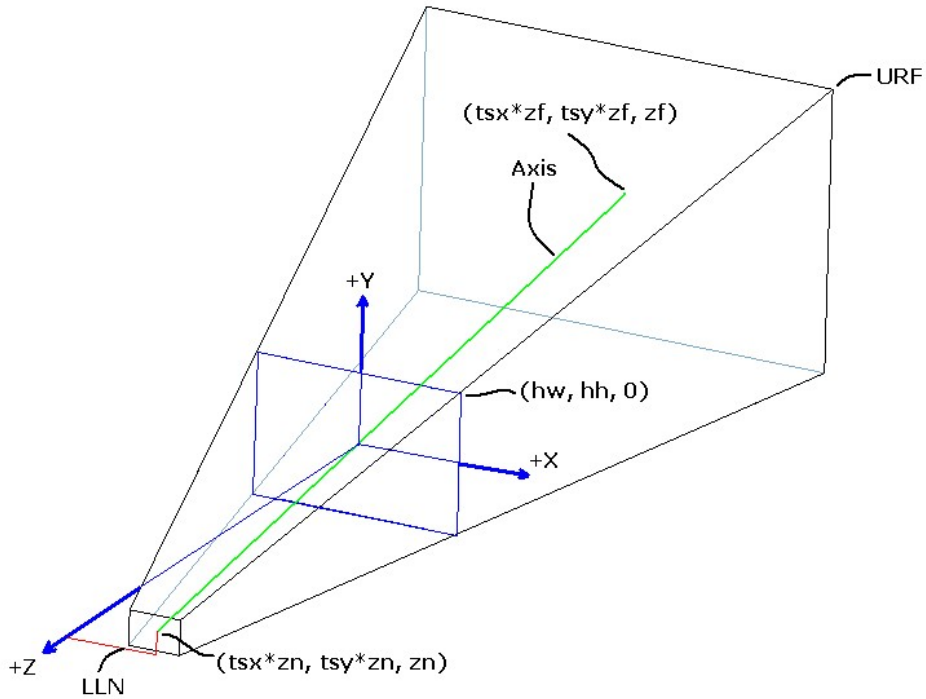
HalfWidth, HalfHeight, ZNear, ZFar, InverseEyeZ, TanSkewX, TanSkewY

These are abbreviated as

hw, *hh*, *zn*, *zf*, *iez*, *tsx* and *tsy*

hw and *hh* define a rectangle in the z = 0 plane. The front and back clipping planes are the z = *zn* and z = *zf* planes. The *tsx* and *tsy* values specify the horizontal and vertical trigonometric tan values of an axis passing through the view space origin and *iez* specifies the inverse of the eyepoint's Z view space coordinate. Right-handed spaces have $zn > zf$ and left-handed ones have $zn < zf$. For perspective views ensure either $1/iez > zn > zf$ or $1/iez < zn < zf$.

Figure 2 shows the general view volume with various coordinates including those for the lower left near (LLN) and upper right far (URF) corners.



$$LLN = (-hw + zn \cdot (tsx + hw \cdot iez) \quad -hh + zn \cdot (tsy + hh \cdot iez) \quad zn)$$
$$URF = (hw + zf \cdot (tsx - hw \cdot iez) \quad hh + zf \cdot (tsy - hh \cdot iez) \quad zf)$$

Figure 2. General view volume.

Nearly all 3D programs use normal or right prisms and frustums where the axis is normal to the rectangular cross section, and so *tsx* and tsy are both zero. Correct stereo viewing requires non-zero *tsx* and *tsy* values, as will be shown later. The use of *ZNear* and *ZFar*, instead of a *HalfDepth* value, disassociates the width and height values from the placement of the near and far clipping planes.

Homogenous coordinates and transformations are almost universally used to transform and clip geometry. The view-to-clip-space 'projection' matrices that transform to either the OpenGL or the Direct3D clipping spaces are:

$$Pgl = \begin{bmatrix} \frac{1}{hw} & 0 & 0 & 0 \\ 0 & \frac{1}{hh} & 0 & 0 \\ \frac{-tsx}{hw} & \frac{-tsy}{hh} & \frac{-(2-(zn+zf)\cdot iez)}{zn-zf} & -iez \\ 0 & 0 & \frac{(zn+zf)-2\cdot zn\cdot zf\cdot iez}{zn-zf} & 1 \end{bmatrix}$$

$$Pd3d = \begin{bmatrix} \frac{1}{hw} & 0 & 0 & 0 \\ 0 & \frac{1}{hh} & 0 & 0 \\ \frac{-tsx}{hw} & \frac{-tsy}{hh} & \frac{-(1-zf\cdot iez)}{zn-zf} & -iez \\ 0 & 0 & \frac{zn\cdot(1-zf\cdot iez)}{zn-zf} & 1 \end{bmatrix}$$

These matrices provide for both parallel and perspective views.

Transforming the lower left front and upper right back view volume corners to the two clipping spaces helps confirm these matrices. These points, as homogenous coordinates, are:

$$LLN = (-hw + zn\cdot(tsx + hw\cdot iez) \quad -hh + zn\cdot(tsy + hh\cdot iez) \quad zn \quad 1)$$
$$URF = (hw + zf\cdot(tsx - hw\cdot iez) \quad hh + zf\cdot(tsy - hh\cdot iez) \quad zf \quad 1)$$

So

$$LLN \times Pgl$$
$$= (-1 + zn\cdot iez \quad -1 + zn\cdot iez \quad -1 + zn\cdot iez \quad 1 - zn\cdot iez) \qquad \rightarrow (-1 \quad -1 \quad -1)$$
$$URF \times Pgl$$
$$= (1 - zf\cdot iez \quad 1 - zf\cdot iez \quad 1 - zf\cdot iez \quad 1 - zf\cdot iez) \qquad \rightarrow (1 \quad 1 \quad 1)$$
$$LLN \times Pd3d$$
$$= (-1 + zn\cdot iez \quad -1 + zn\cdot iez \quad 0 \quad 1 - zn\cdot iez) \qquad \rightarrow (-1 \quad -1 \quad 0)$$
$$URF \times Pd3d$$
$$= (1 - zf\cdot iez \quad 1 - zf\cdot iez \quad 1 - zf\cdot iez \quad 1 - zf\cdot iez) \qquad \rightarrow (1 \quad 1 \quad 1)$$

as required.

When *iez* is zero the eyepoint is at infinity resulting in a parallel projection. A perspective projection has the relationships $1/iez > zn > zf$ or $1/iez < zn < zf$. There is another type of projection called a

hypercentric projection with the relationships $zn > zf > 1/iez$ or $zn < zf < 1/iez$. Although physical lenses** with this projection exist I have never come across this type of projection being used in computer graphics. Notice how viewing a cube face-on in a hypercentric projection exposes five of the cube's six faces.

**See Edmund Optics Hypercentric Lenses
https://www.edmundoptics.com.au/f/hypercentric-lenses/14571/

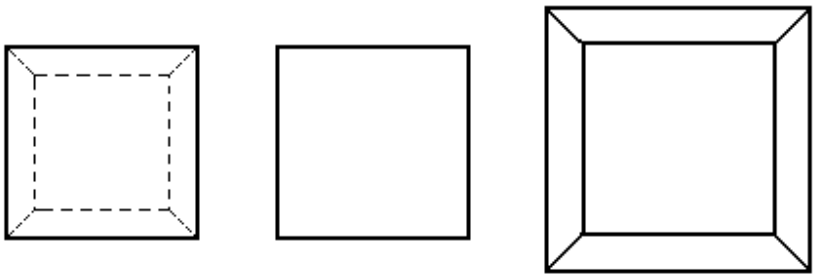


Figure 3. A cube viewed face-on in a perspective, parallel and hypercentric views.

All the historical types of views where straight 3D lines correspond to straight 2D lines are implemented by this general projection transformation.

- Parallel ($iez = 0$)
  - Orthographic ($tsx = 0$ and $tsy = 0$)
    - Top, front, side, plan, elevation, etc.
    - Dimetric
    - Trimetric
    - Isometric
  - Oblique ($tsx \neq 0$ and/or $tsy \neq 0$)
    - Cavalier
    - Cabinet
- Perspective ($1/iez > zn > zf$ or $1/iez < zn < zf$)
  - One point
  - Two point
  - Three point
- Hypercentric ($zn > zf > 1/iez$ or $zn < zf < 1/iez$)

Note: A physical camera lens does not precisely map a 3D straight line to a 2D straight line.

iez is efficient for viewing and motion calculations but it is an awkward value for the user interface. A better user interface value is the 'view angle' defined here to be the smaller of the unskewed horizontal and vertical view volume's taper angles.

$$ViewAngle = 2 \cdot tan^{-1}(min(hw, hh) \cdot |iez|) \cdot \frac{180}{\pi}$$

The use of this view angle parameter will extend wide views sideways and extend tall views vertically to ensure a square prism or frustum corresponding to this angle is always visible. Drag the corner of the *NewView* demo window to see this in action. Changing the view angle is akin to changing the zoom on a physical camera's zoom lens. For most 3D programs, a perspective flag is sufficient to toggle the view angle between zero, for a parallel view, and 45°, for a perspective view.

3D computer graphics programming interfaces, such as OpenGL and Direct3D, follow the computer graphics viewing literature to define perspective and parallel projections as different transformations. This difference is due to the eyepoint being set as the origin. The eyepoint is a projection singularity. As a result, the lower right perspective projection matrix value must be zero, whereas parallel projections require this value to be non-zero. Parallel and perspective projections are actually alike and can be handled with the same viewing code as is clear from the projection matrices shown above. The use of a common set of viewing parameters for both parallel and perspective projections help simplify motion algorithms used to control the view or to move other items.

OpenGL and Direct3D usually require parallel matrices, with the last column being [0, 0, 0, 1], and perspective matrices with the last column being [0, 0, -1, 0]. When iez is zero, the standard form given above has the required parallel last column. For perspective views, the origin needs to be translated to the eyepoint to make the last column [0, 0, -1, 0]. Note: Scaling a homogenous projection matrix by any non-zero value has no effect on the transformation.

Let *ez* = 1/*iez*

$$Pgl = iez \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ \frac{-tsx}{iez} & \frac{-tsy}{iez} & \frac{-1}{iez} & 1 \end{bmatrix} \cdot \begin{bmatrix} \frac{ez}{hw} & 0 & 0 & 0 \\ 0 & \frac{ez}{hh} & 0 & 0 \\ \frac{-ez \cdot tsx}{hw} & \frac{-ez \cdot tsy}{hh} & \frac{-(2 \cdot ez - (zn + zf))}{zn - zf} & -1 \\ 0 & 0 & \frac{-2 \cdot (ez^2 - ez \cdot (zn + zf) + zn \cdot zf)}{zn - zf} & 0 \end{bmatrix}$$

$$Pd3d = iez \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ \frac{-tsx}{iez} & \frac{-tsy}{iez} & \frac{-1}{iez} & 1 \end{bmatrix} \cdot \begin{bmatrix} \frac{ez}{hw} & 0 & 0 & 0 \\ 0 & \frac{ez}{hh} & 0 & 0 \\ \frac{-ez \cdot tsx}{hw} & \frac{-ez \cdot tsy}{hh} & \frac{-(ez - zf)}{zn - zf} & -1 \\ 0 & 0 & \frac{-(ez - zn) \cdot (ez - zf)}{zn - zf} & 0 \end{bmatrix}$$

The sample code in Appendix A uses these formulae to set up the OpenGL and Direct3D transformation matrices given the general viewing parameters. Appendix B is an example showing how to use a useful set of viewing and interaction variables to calculate the general viewing parameters.

Stereo Views

Stereo viewing hardware and software try to mimic real world viewing by mapping a virtual left and a right view volume to the viewer's eyes such that the left and right 2D images coincide in the same

physical space in front of the viewer. The stereo view can be compared with viewing a real world scene through a rectangular hole in the same physical space. Unlike the real world, though, the focal distance is the optical distance of the 2D images, not the distance of the real objects. Also, there are regions in front of the rectangle where graphics is visible to only one eye. In the real world, the object would obstruct the frame of the rectangle, but in the virtual stereo view, the object disappears from one eye's view.

A precise stereo view requires the virtual viewing geometry to match the physical viewing geometry. Only when a physical eye is directly in front of the center of the viewed image should the virtual view frustum be a normal or right frustum. In all other cases, the view frustum should be skewed. Also, there is only one true physical position to view a stereo pair. The human brain compensates for considerable variations between the virtual and physical viewing geometry, which is why hundreds of people can watch a 3D movie.

The virtual distance between the virtual eyepoints in relation to the distance between the viewer's physical eyes scales a scene. For instance, a stereo pair of cameras mounted on an airport terminal 63 m (207') apart will present the airport at $1/1000^{th}$ its size for a viewer with a 63mm (2.48") eye separation. Note the camera lenses axes should be parallel to each other and the camera feeds trimmed accordingly to replicate the geometry of the physical viewer's eyes to the physical stereo screen. Imagine the viewer's eyes and the computer screen being 1,000 times bigger in the airport. Don't be concerned with the misnamed eye direction.

Many stereo viewing texts, articles and implementations incorrectly use right or normal view volumes and rotate them slightly. The result is close to correct so the viewer does see a decent 3D image. Also, almost all stereo viewing implementations have the same screen-to-eye distance for both eyes. If the physical viewer's eye distances can vary then the virtual geometry should not have this restriction.

There is some confusion regarding the term 'eye direction' in the graphics literature as the eye direction is mathematically the normal to the rectangular cross section. To illustrate the misnomer consider viewing a computer screen from well off to one side of the screen. The corresponding 'view direction', being normal to the screen, points towards midair next to the screen and doesn't even pass through the view volume.

A suitable definition for a stereo view uses the same definition of view space as above. The overlapped rectangle is centered on the view space origin. A 3D position for each eye in view space completes the definition.

HalfWidth, HalfHeight, ZNear, ZFar, LeftEye(x,y,z), RightEye(x,y,z)

Immersive Virtual Reality

Systems, such as the CAVE[1] and iCinema[2], use multiple stereo views surrounding the viewer(s) to provide an immersive effect. The CAVE has walls, a floor and a roof. The iCinema has a tall cylindrical screen with twelve cameras projecting as many as 24 stereo pairs. Neighboring stereo pairs are aligned so the edge of one pair closely matches the edge of its neighboring pair. The effect is quite dramatic.

Augmented Reality

Augmented reality is the combining of real-world images with corresponding virtual world images. The virtual view volume geometry must match the physical viewing geometry. Augmented reality can

provide the effect of X-ray vision to 'see', for instance, wires in the wall of an aircraft. Finished buildings could be shown overlaid on a direct view of a live construction site.

The use of a head mounted display with a camera and a head tracker can produce stunning dynamic X-ray vision results, delivering significant benefits to, for instance, a repair technician. Robotic surgery has progressed to the point where surgeons are able to use cameras and robots to perform surgery while being physically remote from the patient. 3D computer graphic imagery adds details not visible to the camera. Augmented reality with live scanning technology could deliver the equivalent of X-ray vision where a surgeon can graphically see the internals of the patient overlaid on their real-world view of the patient.

It is amazing to realize the first head mounted 3D display with a head tracker was built in the mid 1960's by the computer graphics pioneer Ivan Sutherland. This is one technology that has taken literally decades to go from invention to commercialization.

Using the Code

The code to configure OpenGL and Direct3D is in the `ConfigureView_OpenGL()` and `ConfigureView_Direct3D()` functions in the *GeneralView.cpp* file. These functions are declared in *GeneralView.h* as:

C++
```
// GeneralView.h

void ConfigureView_OpenGL(float ViewVolume[7], float ViewToWorld[4][3]);
#ifdef _D3D9_H_
void ConfigureView_Direct3D(FLOAT ViewVolume[7], FLOAT ViewToWorld[4][3],
    D3DMATRIX& Projection, D3DMATRIX& World);
#endif
```

Either copy the code or the GeneralView.* files over to your project. These files were written to be independent of other code, like the SpatialMath module. To display an object centered on the world space origin within a sphere of radius *Radius*, use:

C++
```
      ViewVolume[0] = 0.6f * Radius;  // view rectangle HalfWidth
      ViewVolume[1] = 0.6f * Radius;  // view rectangle HalfHeight
      if (PixelsAcross >= PixelsUpDown) // window width and height
            ViewVolume[0] *= (float)PixelsAcross / PixelsUpDown;
      else
            ViewVolume[1] *= (float)PixelsUpDown / PixelsAcross;
      ViewVolume[2] = 4.0f * Radius;  // near clipping plane z-coord
      ViewVolume[3] = -4.0f * Radius;  // far clipping plane z-coord
      ViewVolume[4] = 0;               // parallel view
      ViewVolume[5] = ViewVolume[6] = 0;  // no skew

float ViewToWorld[4][3] = { {1,0,0},{0,1,0},{0,0,1},{0,0,0} };
```

Then, call `ConfigureView_OpenGL()` or `ConfigureView_Direct3D()`.

Following `ConfigureView_Direct3D()` is the sample code with a set of high-level viewing parameters suitable for common 3D programs. This can be used as a basis for the 3D interaction section of the user interface.

The demo source code shows how to use mouse interaction although it is not explained here.

Conclusion

Approaching 3D computer graphics viewing as a mapping of a 3D view volume to a 2D image leads to a very straight forward and comprehensive set of viewing parameters compared with the traditional approaches found in most computer graphics textbooks.

Today viewing transformations are implemented in a myriad of ways yet all viewing essentially maps a 3D view volume to a 2D image. The lack of a standard set of viewing parameters leads to increased software development and support costs, inconsistent user interfaces, the lack of features such as stereo and a significant development barrier to the integration of new types of input devices. A consistent set of viewing parameters would provide the foundation needed to standardize viewing and motion implementations as well as provide consistent and rich user interfaces for 3D viewing and interaction. The set of seven viewing parameters along with a rotation and translation as described here is comprehensive, simple and flexible enough to satisfy all the needs of 3D viewing, motion and interaction.

References

[1] en.wikipedia.org/wiki/Cave_Automatic_Virtual_Environment
[2] www.icinema.unsw.edu.au
[3] Foley, Jim, Andy van Dam, Steve Feiner and John Hughes. *Computer Graphics: Principles and Practice.* 2nd Ed. 1990
[4] Lengyel, Eric. *Mathematics for 3D Game Programming & Computer Graphics.* 2nd Ed. 2004
[5] Edmund Optics Hypercentric Lenses - https://www.edmundoptics.com.au/f/hypercentric-lenses/14571/

Notices

Copyright © 2009-2026 John Hilton. All rights reserved.

Trademarks and Registered Trademarks used in this document are the property of their respective owners.

Appendix A

The following code uses the general viewing parameters and the ViewToWorld rotate/translate transformation to configure OpenGL and Direct3D.

```
void ConfigureView_OpenGL(float ViewVolume[7], float ViewToWorld[4][4])
{
    // Set up the GL_PROJECTION and GL_MODELVIEW matrices which transform
    // from world space to clip space. Parallel and perspective views are
    // handled along with left or right handed spaces.
    // ViewToWorld must only be a rotation and translation.
    //      WorldToClip = ModelView * Projection
    //                  = Inverse(ViewToWorld) * ViewToClipGL(ViewVolume)
    // ViewToClipGL = [    1/hw       0                           0    0 ]
    //                [       0    1/hh                           0    0 ]
    //                [ -tsx/hw -tsy/hh    -(2-(zn+zf)*iez)/(zn-zf) -iez ]
    //                [       0       0 (zn+zf-2*zn*zf*iez)/(zn-zf)    1 ]
    // Abbreviations stand for
    //          HalfWidth, HalfHeight, ZNear, ZFar
    //          InverseEyeZ, TanSkewX, TanSkewY
    // Note: OpenGL's perspective view space has a different origin
    // to ViewToWorld's and ViewToClip's view space origin.

    // Use references for code readability
    struct TViewVolume { FLOAT hw, hh, zn, zf, iez, tsx, tsy; };
    TViewVolume& vv = *(TViewVolume*)ViewVolume;
    float& hw = vv.hw, & hh = vv.hh, & zn = vv.zn, & zf = vv.zf;
    float& iez = vv.iez, & tsx = vv.tsx, & tsy = vv.tsy;

    glMatrixMode(GL_PROJECTION);
    static const float kVerySmall = 1e-6f;
    if (abs(iez) < kVerySmall)
    {
        // Orthographic view
        // Create a matrix with the 4th column [0, 0, 0, 1]
        GLfloat Ortho[4][4] = {
            {    1 / hw,       0,               0, 0 },
            {       0,    1 / hh,               0, 0 },
            { -tsx / hw, -tsy / hh,      -2 / (zn - zf), 0 },
            {       0,       0, (zn + zf) / (zn - zf), 1 }
        };
        glLoadMatrixf(Ortho[0]);
        glMatrixMode(GL_MODELVIEW);
        glLoadIdentity();
    }
    else
    {
        // Perspective view
        // Create a perspective matrix with the 4th column [0, 0, 1, 0]
        // which has the origin at the eyepoint.
        float ez = 1 / iez;   // The eyepoint's view space Z coordinate
        GLfloat Persp[4][4] = {
            {    -ez / hw,         0,                                  0, 0 },
            {         0,    -ez / hh,                                  0, 0 },
            { ez * tsx / hw, ez * tsy / hh,            (2 * ez - (zn + zf)) / (zn - zf), 1
},
            {         0,         0, 2 * (ez * (ez - (zn + zf)) + zn * zf) / (zn - zf), 0 }
        };
        glLoadMatrixf(Persp[0]);
        glMatrixMode(GL_MODELVIEW);
```

```
        glLoadIdentity();
        glTranslatef(-ez * tsx, -ez * tsy, -ez);
    }

    // Do the inverse of the ViewToWorld rotation and translation
    // Transpose the rotation to invert it
    GLfloat WorldToViewRotate[4][4] = {
        {   ViewToWorld[0][0], ViewToWorld[1][0], ViewToWorld[2][0], 0 },
        {   ViewToWorld[0][1], ViewToWorld[1][1], ViewToWorld[2][1], 0 },
        {   ViewToWorld[0][2], ViewToWorld[1][2], ViewToWorld[2][2], 0 },
        {                   0,                 0,                 0, 1 }
    };
    glMultMatrixf(WorldToViewRotate[0]);
    glTranslatef(-ViewToWorld[3][0], -ViewToWorld[3][1], -ViewToWorld[3][2]);
}

void ConfigureView_Direct3D(FLOAT ViewVolume[7], FLOAT ViewToWorld[4][4],
    D3DMATRIX& Projection, D3DMATRIX& View)
{
    // Set up the Projection and View matrices which transform from
    // world space to clip space. Parallel and perspective views are
    // handled along with left or right handed spaces.
    // ViewToWorld must only be a rotation and translation.
    //   WorldToClip = View * Projection
    //               = Inverse(ViewToWorld) * ViewToClipD3D(ViewVolume)
    //   ViewToClipD3D = [    1/hw       0                     0    0 ]
    //                   [       0    1/hh                     0    0 ]
    //                   [ -tsx/hw -tsy/hh   -(1-zf*iez)/(zn-zf) -iez ]
    //                   [       0       0 zn*(1-zf*iez)/(zn-zf)    1 ]
    // Abbreviations stand for
    //          HalfWidth, HalfHeight, ZNear, ZFar
    //          InverseEyeZ, TanSkewX, TanSkewY
    // Note: Direct3D's perspective view space has a different origin
    // to ViewToWorld's and ViewToClip's view space origin.

    // Use references for code readability
    struct TViewVolume { FLOAT hw, hh, zn, zf, iez, tsx, tsy; };
    TViewVolume& vv = *(TViewVolume*)ViewVolume;
    float& hw = vv.hw, & hh = vv.hh, & zn = vv.zn, & zf = vv.zf;
    float& iez = vv.iez, & tsx = vv.tsx, & tsy = vv.tsy;

    // Set View = Inverse(ViewToWorld)
    //          = XFRM(-ViewToWorld.trn) * XFRM(Transpose(ViewToWorld.rot))
    //          = [      1      0      0 0 ] * [ rot00 rot10 rot20 0 ]
    //            [      0      1      0 0 ]   [ rot01 rot11 rot21 0 ]
    //            [      0      0      1 0 ]   [ rot02 rot12 rot22 0 ]
    //            [ -trn.x -trn.y -trn.z 1 ]   [     0     0     0 1 ]
    // where trn is the ViewToWorld.m[3][0] to m[3][2] vector and rot is the
    // ViewToWorld.m[0][0] to m[2][2] matrix.
    for (int i = 0; i < 3; i++)
    {
        // Transform -ViewToWorld.trn from world space to view space
        View.m[3][i] = -ViewToWorld[3][0] * ViewToWorld[i][0]
            - ViewToWorld[3][1] * ViewToWorld[i][1]
            - ViewToWorld[3][2] * ViewToWorld[i][2];
        // Invert ViewToWorld.rot
        View.m[0][i] = ViewToWorld[i][0];
        View.m[1][i] = ViewToWorld[i][1];
        View.m[2][i] = ViewToWorld[i][2];
        // Zero m[0][3] to m[2][3]
        View.m[i][3] = 0;
```

```
    }
    View.m[3][3] = 1;

    static const float kVerySmall = 1e-6f;
    if (abs(iez) < kVerySmall)
    {
        // Orthographic view
        // Create a matrix with the 4th column [0, 0, 0, 1]
        D3DXMATRIX Ortho(1 / hw, 0, 0, 0,
            0, 1 / hh, 0, 0,
            -tsx / hw, -tsy / hh, -1 / (zn - zf), 0,
            0, 0, zn / (zn - zf), 1);
        Projection = Ortho;
    }
    else
    {
        // Perspective view
        // Create a perspective matrix with the 4th column [0, 0, 1, 0]
        // which has the origin at the eyepoint.
        FLOAT ez = 1 / iez;   // The eyepoint's view space Z coordinate
        D3DXMATRIX Persp(-ez / hw, 0, 0, 0,
            0, -ez / hh, 0, 0,
            ez * tsx / hw, ez * tsy / hh, (ez - zf) / (zn - zf), 1,
            0, 0, (ez - zn) * (ez - zf) / (zn - zf), 0);
        Projection = Persp;

        // Add the translation from the eyepoint to the view space origin
        View.m[3][0] += -ez * tsx;
        View.m[3][1] += -ez * tsy;
        View.m[3][2] += -ez;
    }
}
```

Appendix B

The following code is based on a set of high-level viewing parameters useful for many 3D programs.

```
// The following variables are the interface between the interactive user
// interface and the viewing code.
float g_ViewOrientation[3][3];    // orients the view in world space about
// ViewCenter
float g_ViewCenter[3];            // a world space point mapped to the center
// of the 2D image
float g_ViewSize;                 // minimum width or height of the view's
// cross section at ViewCenter
float g_ViewAngle;                // either
//    0 - a parallel view
//    45*M_PI/180 - a perspective view
float g_BoundingSphere[4];        // world space x, y, z and radius, bounds the
// world space region of renderable geometry
int   g_ViewportWidth, g_ViewportHeight;    // in pixels

// Declare a 3D vector type
union TVec3 { float v3[3]; struct { float x, y, z; }; };

// Provide an inverse rotate/translate transformation
void Vec3MultInverseRotTrn(float vout[3], float v3[3], float RotTrn[4][4])
{
    float tmp[3] = {
        v3[0] - RotTrn[3][0],
        v3[0] - RotTrn[3][1],
        v3[0] - RotTrn[3][2]
    };
    for (int i = 0; i < 3; i++)
        vout[i] = tmp[0] * RotTrn[i][0]
        + tmp[1] * RotTrn[i][1]
        + tmp[2] * RotTrn[i][2];
}

// Various declarations
bool UsingOpenGL;
void ConfigureView_OpenGL(float ViewVolume[7], float ViewToWorld[4][4]);
void ConfigureView_Direct3D(FLOAT ViewVolume[7], FLOAT ViewToWorld[4][4],
    D3DMATRIX& Projection, D3DMATRIX& View);
D3DXMATRIXA16 g_projmat, g_viewmat;

void ConfigureView()
{
    // Create the ViewToWorld transformation by combining g_ViewOrientation
    // and g_ViewCenter.
    float ViewToWorld[4][4];
    for (int i = 0; i < 3; i++)
        *(TVec3*)ViewToWorld[i] = *(TVec3*)g_ViewOrientation[i];
    *(TVec3*)ViewToWorld[3] = *(TVec3*)g_ViewCenter;
    ViewToWorld[3][0] = ViewToWorld[3][1] = ViewToWorld[3][2] = 0;
    ViewToWorld[3][2] = 1;

    // Create the ViewVolume
    //    HalfWidth, HalfHeight, ZNear, ZFar, InverseEyeZ, TanSkewX, TanSkewY
    union TViewVolume {
        float v7[7];
        struct { float hw, hh, zn, zf, iez, tsx, tsy; };
    } vv;
```

```
    vv.tsx = vv.tsy = 0;    // not using skewed views
    vv.hw = vv.hh = 0.5f * g_ViewSize;
    // Increase the wider dimension
    if (g_ViewportWidth >= g_ViewportHeight)
        vv.hw *= (float)g_ViewportWidth / g_ViewportHeight;
    else
        vv.hh *= (float)g_ViewportHeight / g_ViewportWidth;

    // Map the bounding sphere from world space to view space
    TVec3 SphereCenterView;
    Vec3MultInverseRotTrn(SphereCenterView.v3, g_BoundingSphere, ViewToWorld);

    float& radius = g_BoundingSphere[3];
    vv.zn = SphereCenterView.x + radius;
    vv.zf = SphereCenterView.x - radius;
    static const float kVerySmall = 1e-6f;
    if (g_ViewAngle < kVerySmall)
        vv.iez = 0;     // parallel view
    else
    {
        // Perspective view
        // Ensure the near clipping plane is not too close to the eyepoint
        // and the ratio between the near and far clipping plane distances
        // provides adequate resolution for the z-buffer.
        float EyeZ = min(vv.hw, vv.hh) / tan(0.5f * g_ViewAngle);
        static const float kMinNearDistance = 1e-4f;
        if (vv.zn > EyeZ - kMinNearDistance)
            vv.zn = EyeZ - kMinNearDistance;
        static const float kMinNearFarFactor = 5e-4f;
        if (vv.zn > (EyeZ - vv.zf) * kMinNearFarFactor)
            vv.zn = (EyeZ - vv.zf) * kMinNearFarFactor;
        vv.iez = 1 / EyeZ;
    }

    // Configure the view
    if (UsingOpenGL)
        ConfigureView_OpenGL(vv.v7, ViewToWorld);
    else
        ConfigureView_Direct3D(vv.v7, ViewToWorld, g_projmat, g_viewmat);
}
```